Digital Systems – Notes

1. Basics of Digital Systems.

Answer: A digital system processes signals that are discrete in time and represented by binary numbers. Most real signals (voice, video, radar, sensor outputs) are analog, so they are first converted to digital form, processed by a processor or computer, and converted back to analog if needed. A typical system has an anti-aliasing filter, an A/D converter, a digital processor, a D/A converter and a reconstruction filter.

Digital processing has many advantages: it is less sensitive to temperature, aging and component values; the accuracy can be chosen by the number of bits; signals can be stored easily; one processor can be shared between several signals; and the system can be changed just by changing the program. The disadvantages are that we need A/D and D/A converters, the frequency range is limited by the sampling rate, and quantization errors appear.

2. Digital systems based on computer engineering complexes.

Answer: A computer engineering complex is a set of hardware and software parts that work together as one system. The main parts are the processor (CPU, microcontroller or DSP), memory, input/output units, A/D and D/A converters, and the buses that connect them, plus the software that runs on them. A digital system built on such a complex takes data from sensors, processes it with a program, and gives the result to the user or to actuators.

A good example is the onboard digital computer complex of an aircraft, like the Flight Management System and the digital flight control system. Sensors give analog signals, the A/D converter changes them to numbers, the processor runs the control algorithm, and the D/A converter sends the command to the actuators. Using several processors together (multiprocessing) increases the speed and the reliability.

3. Digital system and its fundamental principles of operation.

Answer: A digital system works with information that is written as binary numbers (0 and 1) and processes it with logic circuits and processors. The input is a sequence x[n] and the output is a sequence y[n], and the system is described by a rule (a difference equation) between them.

The fundamental principles are these:

- The analog signal is sampled at regular intervals and quantized to a finite number of bits.

- The processing is done on numbers using only three basic operations: addition, multiplication by a constant, and unit delay.
- All operations are synchronized by a clock, so the result is repeatable and does not drift.
- If an analog output is needed, the result goes through a D/A converter and a smoothing filter.

4. Signal reconstruction from discrete samples.

Answer: If the sampling theorem is satisfied ($f_s \geq 2 \cdot f_{max}$), the original signal $x(t)$ can be recovered completely from its samples $x(nT)$. In theory this is done by passing the samples through an ideal low-pass filter with cutoff $f_s/2$, which is the same as sinc interpolation: $x(t) = \Sigma\, x(nT) \cdot \text{sinc}((t - nT)/T)$.

An ideal filter cannot be built, so in practice the D/A converter holds each sample constant for one sampling interval (zero-order hold). This gives a staircase signal, and then an analog smoothing low-pass filter (anti-imaging filter) removes the extra high-frequency images around multiples of f_s. If f_s is too low, aliasing happens and the signal cannot be reconstructed correctly.

5. Data conversion and quantization.

Answer: Data conversion means changing the signal between analog and digital form: A/D conversion at the input and D/A conversion at the output. A/D conversion has three steps: sampling (reduces the time information), quantization (reduces the amplitude information) and coding (writes the level as a binary number).

Quantization means that each sample is replaced by one of a finite number of levels, $L = 2^N$, where N is the number of bits. The step size is $\Delta = R / 2^N$, where R is the full-scale range. The error is at most $\Delta/2$. More bits give a smaller error and better resolution, but need more memory and a more expensive converter.

Example: a pressure sensor gives 0–3 V and we use 3 bits. Then $\Delta = 3/8 = 0.375$ V and the maximum error is 0.1875 V.

6. Sample-and-hold circuits.

Answer: A sample-and-hold (S/H) circuit takes the value of the analog signal at regular intervals T, called the sampling period, and holds it constant for the rest of the interval. After a short acquisition time, in which the sample is taken, the circuit keeps the value steady. The hold time is needed because the A/D converter needs some time to produce the digital code that matches the sample.

If x(t) is the input, the output is x(nT), and fs = 1/T is the sampling frequency. After sampling the signal is discrete in time but still continuous in amplitude. In practice an S/H circuit is made from an electronic switch, a hold capacitor and a buffer amplifier.

7. Digital-to-analog conversion.

Answer: After digital processing is finished, the digital signal must be changed back to analog. The D/A converter changes each digital code into an analog voltage that is proportional to the number of the code. This voltage is held constant by a zero-order hold until the next code comes, one sampling interval later. This gives a staircase signal, which has frequencies above the signal band.

In the frequency domain these high frequencies are images (copies) of the original spectrum around multiples of fs. An analog low-pass smoothing filter, called the anti-imaging or reconstruction filter, removes them, and only the baseband from 0 to fs/2 stays. For example, a 3-bit D/A converter has 8 possible output levels.

8. Analog-to-digital conversion

Answer: A/D conversion changes an analog signal, which is continuous in time and amplitude, into a digital signal. It is a three-step process:

1. Sampling (sample-and-hold): the signal is measured at regular intervals T, so we get a discrete-time signal.
2. Quantization: each sample is rounded to one of $L = 2^N$ levels, so we get a discrete-amplitude signal.
3. Coding: each level is written as an N-bit binary code.

Before sampling, an analog anti-aliasing low-pass filter removes frequencies above fs/2. The resolution of the converter is $\Delta = R / 2^N$. A common fast type is the flash ADC, which uses many comparators and converts in one step.

9. Discretization of continuous-time processes.

Answer: Discretization means replacing a continuous-time process x(t) by a sequence of its values x(nT), and replacing its differential equation by a difference equation, so that a computer can handle it. The sampling period T must be chosen small enough, according to the sampling theorem.

The derivative can be approximated by a difference. With the backward difference, $dy/dt \approx (y[n] - y[n-1]) / T$, which means $s \approx (1 - z^{-1}) / T$. Other methods are the forward difference, the bilinear (Tustin) transform $s = (2/T)\cdot(1 - z^{-1})/(1 + z^{-1})$, and the step-invariant method (sampling with a zero-order hold).

Example: for dy/dt = −a·y + x the backward difference gives y[n] − y[n−1] = T·(−a·y[n] + x[n]), which can be solved for y[n] and programmed directly.

10. Reconstruction of continuous functions.

Answer: Reconstruction means creating a continuous function x(t) from the samples x[n]. There are three common ways. The ideal way is sinc interpolation, which is exact when fs ⩾ 2·fmax. The most common practical way is the zero-order hold, which keeps each value constant for T seconds, and its transfer function is $(1 - e^{-sT}) / s$. A better way is the first-order hold, which connects the samples with straight lines.

In each case the result still contains spectral images, so a smoothing analog low-pass filter is used at the end. If the signal was sampled below the Nyquist rate, the original function cannot be restored, because aliasing already mixed the frequencies.

11. Fourier transform.

Answer: The Fourier transform shows which frequencies are inside a signal. For a continuous signal, $X(f) = \int x(t)\cdot\exp(-j2\pi ft)\, dt$. For a discrete-time signal we use the DTFT:

$X(\Omega) = \Sigma\, x[n]\cdot\exp(-j\Omega n)$, where $\Omega = 2\pi f/fs$ is the digital frequency in radians.

The DTFT is periodic with period 2π, so we only need Ω from 0 to π. A delay of M samples in time becomes a multiplication by exp(−jΩM) in frequency. For a finite signal of N samples we use the DFT, $X[k] = \Sigma\, x[n]\cdot\exp(-j2\pi kn/N)$, which can be calculated quickly with the FFT. For a system, the frequency response H(Ω) is the DTFT of the impulse response h[n].

12. Modified Z-transform.

Answer: The ordinary z-transform gives information only at the sampling instants t = nT. The modified z-transform is used when we also need the values between the samples, or when the system has a time delay that is not a whole number of sampling periods. We add a parameter m, with 0 ⩽ m ⩽ 1, which shifts the sampling instants by a fraction of the period:

$F(z, m) = \Sigma\, f(nT + mT)\cdot z^{-n}$

Example: for $f(t) = e^{-at}$, $F(z, m) = e^{-amT} / (1 - e^{-aT}\cdot z^{-1})$. When m = 0 we get the ordinary z-transform. It is useful for finding the hidden ripple between samples in sampled-data control systems.

13. Analog model of the extrapolator.

Answer: The extrapolator (hold element) is the part of the D/A converter that builds a continuous signal from the samples, using the past values. The zero-order extrapolator holds the last sample constant: $y(t) = x(nT)$ for $nT \leqslant t < (n+1)T$. Its impulse response is a rectangular pulse of length T, so the analog model (transfer function) is

$G(s) = (1 - e^{-sT}) / s$

Its frequency response is $T \cdot \text{sinc}(fT)$ with a phase of $-\omega T/2$, so it works like a low-pass filter and adds an average delay of T/2 to the signal. This delay reduces the phase margin of a control loop. The first-order extrapolator uses the two last samples to continue the signal along a straight line, and it has the transfer function $(1 + Ts) \cdot ((1 - e^{-sT}) / (Ts))^2$.

14. Stable and unstable systems (15 bal)

Answer: A system is stable if every bounded input gives a bounded output (BIBO stability). If a bounded input can give an output that grows without limit, the system is unstable. A sine wave or a step is a bounded input, and a ramp is an unbounded input.

Stability depends on the poles of the transfer function:

- Continuous-time (s-plane): all poles in the left half plane means stable. A pole in the right half plane means unstable, and the output keeps increasing. Simple poles on the imaginary axis give a critically (marginally) stable system, whose output does not decay to zero and does not go to infinity, it just oscillates or stays constant.
- Discrete-time (z-plane): all poles inside the unit circle means stable. A pole outside the unit circle means unstable. A simple pole on the unit circle means marginally stable.

Example: $H(z) = 1/(1 - 0.5 \cdot z^{-1})$ has a pole at 0.5, so it is stable. $H(z) = 1/(1 - 2 \cdot z^{-1})$ has a pole at 2, so it is unstable, because its impulse response 2^n grows. Feedback can also change stability: if the loop gain is increased too much, a stable system can become unstable.

15. 2. Design of Digital Filters BIBO

Answer: Designing a digital filter means finding a transfer function H(z) (or its coefficients) that satisfies the given specifications. The steps are:

1. Set the specifications: type (low-pass, high-pass, band-pass, band-stop), cutoff frequencies, passband ripple, stopband attenuation, and sampling frequency.
2. Choose FIR or IIR. FIR is always stable and can have linear phase, but needs a higher order. IIR needs a lower order, but can be unstable and has nonlinear phase.
3. Calculate the coefficients. For IIR we usually design an analog prototype (Butterworth, Chebyshev) and convert it with the bilinear transform. For FIR we can use the window method.

4. Check stability and the response, then choose a structure (direct, cascade, parallel) and check the finite word length effects.

BIBO (bounded-input bounded-output) stability of a designed filter means that $\Sigma |h[n]| < \infty$. For a causal IIR filter this is true when all poles are inside the unit circle, and FIR filters are always BIBO stable.

16. Stability of Discrete Time Systems Routh-Hurwitz Criterion for stability (10 bal)

Answer: A discrete-time LTI system is stable if all the roots of the characteristic polynomial D(z) (the poles) are inside the unit circle. To test this we can use the Jury test directly in the z-plane, or the Routh–Hurwitz criterion after a transformation.

Routh–Hurwitz criterion. It tells how many roots of a polynomial are in the right half plane without calculating the roots. We write the polynomial $a_n \cdot s^n + \ldots + a_0$ and build the Routh array. The system is stable if there are no sign changes in the first column. The number of sign changes is the number of roots in the right half plane. A necessary condition is that all coefficients exist and have the same sign.

Example 1: $s^3 + 2s^2 + 3s + 4$. The rows are: s^3: 1, 3; s^2: 2, 4; s^1: $(2 \cdot 3 - 1 \cdot 4)/2 = 1$; s^0: 4. The first column 1, 2, 1, 4 has no sign change, so the system is stable.

Example 2: $s^3 + s^2 + 2s + 8$. The s^1 row is $(1 \cdot 2 - 8)/1 = -6$, so the first column is 1, 1, −6, 8. There are two sign changes, so two roots are in the right half plane and the system is unstable.

For discrete-time systems the stable region is the unit circle, so we first use the bilinear transformation $z = (1 + w)/(1 - w)$. It maps the inside of the unit circle to the left half of the w-plane, and then we apply Routh–Hurwitz to the polynomial in w.

Example: $D(z) = z^2 - 1.2z + 0.32$ becomes $2.52w^2 + 1.36w + 0.12$. All coefficients are positive, so it is stable. This agrees with the roots 0.8 and 0.4, which are inside the unit circle. (For a second-order polynomial $a_2 \cdot z^2 + a_1 \cdot z + a_0$ with $a_2 > 0$, the Jury conditions are $D(1) > 0$, $D(-1) > 0$ and $|a_0| < a_2$.)

17. Sampling and quantization. Convolution.

Answer: Sampling takes the values of x(t) at $t = nT$ and reduces the time information, so we get a discrete-time signal. The sampling frequency must satisfy $f_s \geqslant 2 \cdot f_{max}$. **Quantization** rounds each sample to one of 2^N levels with step $\Delta = R/2^N$ and reduces the amplitude information, so we get a digital signal. The quantization error is at most $\Delta/2$.

Convolution. The output of an LTI system is the convolution of the input with the impulse response:

$y[n] = \Sigma\, x[k] \cdot h[n - k]$

To calculate it, we flip h, shift it, multiply it with x point by point, and sum. If x has N_1 samples and h has N_2 samples, the output has $N_1 + N_2 - 1$ samples. Convolution in time becomes multiplication in the z-domain, $Y(z) = H(z) \cdot X(z)$.

Example: x = {1, 2} and h = {1, 1, 1} give y = {1, 3, 3, 2}.

18. Nyquist criterion for stability

Answer: The Nyquist criterion finds the stability of the closed-loop system from the open-loop frequency response $F(j\omega)$. The Nyquist diagram is the polar plot of $F(j\omega)$ for ω from $-\infty$ to $+\infty$. The negative frequency part is the mirror image of the positive part about the real axis. If F(s) has poles on the imaginary axis (for example an integrator), the plot is closed with arcs of infinite radius.

The key point is the critical point (−1, 0). The closed-loop system has poles on the imaginary axis if and only if the plot passes through (−1, 0).

Let nF+ be the number of open-loop poles with positive real part, nW+ the number of closed-loop poles with positive real part, and Ncc the number of counter-clockwise encirclements of (−1, 0). From Cauchy's principle of argument, Ncc = nF+ − nW+.

Criterion: the unit negative feedback system is asymptotically stable if and only if (i) the plot does not pass through (−1, 0), and (ii) Ncc = nF+. If the open-loop system has no poles in the right half plane (nF+ = 0), the condition is simple: the plot must not encircle (−1, 0). If the condition is not satisfied, the closed-loop system has nW+ = nF+ − Ncc unstable poles.

Stability margins. The closer the plot is to (−1, 0), the closer the system is to instability.

- Gain margin: $GM = 1/|F(j\omega\pi)|$, where ωπ is the frequency at which the phase is −180°. It shows how much the gain can be increased before instability. If GM is less than 1 (negative in dB), the system is unstable.
- Phase margin: $PM = 180° + \angle F(j\omega c)$, where ωc is the gain crossover frequency at which |F| = 1. It shows how much additional phase lag the system can tolerate. If PM is negative, the system is unstable.
- A time delay e^{-sT} does not change the magnitude, but adds the phase lag −ωT, so it can easily make the system unstable. The delay margin is DM = PM/ωc.

Example from the lecture: $G(s) = 2/(s + 1)^2 \cdot e^{-s}$ with proportional control. At ωpc = 1.31 rad/s the magnitude is 0.735, so GM = 1/0.735 = 1.36 (2.63 dB). At ωgc = 1 rad/s the phase is −147.3°, so PM = 32.7° (0.57 rad), and the delay margin is 0.57/1 = 0.57 s. Both margins are positive, so the system is stable.

19. Digital controller. D/A conversion

Answer: A digital controller is a computer or microcontroller that calculates the control signal using a difference equation instead of an analog circuit. In a digital control loop the sensor signal goes through an A/D converter, the controller calculates the control signal from the error $e[k] = r[k] - y[k]$, and the result goes through a D/A converter to the actuator and the plant.

A common example is the digital PID controller:

$$u[k] = u[k-1] + Kp\cdot(e[k] - e[k-1]) + Ki\cdot T\cdot e[k] + (Kd/T)\cdot(e[k] - 2e[k-1] + e[k-2])$$

The D/A converter changes the number $u[k]$ into a voltage and a zero-order hold keeps it constant during the sampling interval. This hold adds an average delay of $T/2$, which reduces the phase margin, so the sampling period T must be chosen small enough for the loop to stay stable.

20. Realization of Digital Systems. IIR - Direct, parallel and cascaded realizations

Answer: A realization (structure) is a block diagram of a system made from delays, adders and multipliers, which implements its difference equation. The structure is chosen to reduce computational complexity, memory, and finite word length effects.

- **Direct form I:** the feedforward part (zeros) and the feedback part (poles) each have their own delay elements.
- **Direct form II:** the two parts share the same delays, so it needs only N delays for an order-N filter (a canonical form). It has the same transfer function as form I but uses less memory.
- **Cascade form:** $H(z) = H_1(z)\cdot H_2(z)\cdots H_k(z)$, a product of second-order sections.
- **Parallel form:** $H(z) = C + H_1(z) + H_2(z) + \ldots + H_k(z)$, a sum of second-order sections found by partial fraction expansion.

The cascade and parallel forms are less sensitive to coefficient quantization than a high-order direct form, because the poles are controlled by the small sections one by one.

21. Transfer functions

Answer: The transfer function of a discrete-time system is the ratio of the z-transform of the output to the z-transform of the input with zero initial conditions: $H(z) = Y(z)/X(z)$. It is also the z-transform of the impulse response $h[n]$. For a difference equation it is a ratio of two polynomials in z^{-1}:

$$H(z) = (b_0 + b_1\cdot z^{-1} + \ldots + bM\cdot z^{-M}) / (1 + a_1\cdot z^{-1} + \ldots + aN\cdot z^{-N})$$

The roots of the numerator are the zeros and the roots of the denominator are the poles. If we put $z = \exp(j\Omega)$ we get the frequency response $H(\Omega)$. When two systems are connected in cascade, the transfer functions are multiplied, $H = H_1 \cdot H_2$, and when they are connected in parallel, they are added, $H = H_1 + H_2$.

Example: $y[n] = 0.5 \cdot y[n-1] + x[n]$ gives $H(z) = 1/(1 - 0.5 \cdot z^{-1}) = z/(z - 0.5)$, with a pole at 0.5 and a zero at 0.

22. Modulation and demodulation

Answer: Modulation means changing one parameter of a high-frequency carrier signal (amplitude, frequency or phase) according to the message signal, so that the message can be sent through a channel. After modulation the message spectrum is moved to the carrier frequency fc.

- AM (amplitude modulation): $s(t) = [1 + m \cdot x(t)] \cdot \cos(2\pi \cdot fc \cdot t)$.
- FM (frequency modulation): the instantaneous frequency changes with the message.
- PM (phase modulation): the phase of the carrier changes with the message.
- Digital modulation: ASK, FSK and PSK change the amplitude, frequency or phase of the carrier between a few fixed values.

Demodulation is the opposite process, in which the message is recovered from the received signal. For AM we can use an envelope detector, or coherent demodulation: multiply the received signal by the carrier and use a low-pass filter. For FM we use a discriminator. In digital systems modulation is a multiplication of sequences, and sampling itself can be seen as a modulation (multiplication by an impulse train).

23. Coefficient quantization effects in digital filters

Answer: In a real digital system the filter coefficients are stored with a finite number of bits, so they must be rounded. The rounded coefficients are a little different from the designed ones, so the poles and zeros move to new places and the frequency response is different from the design: the cutoff frequency shifts, the stopband attenuation gets worse, and the ripple increases.

The effect is the strongest when the poles are close to the unit circle. A small error can move a pole outside the circle and make the filter unstable. A high-order filter in direct form is the most sensitive, because one coefficient error changes all the poles together. To reduce the problem we can use more bits, or use cascade or parallel forms of second-order sections. FIR filters with symmetric coefficients keep their linear phase even after quantization, and they are free of limit cycle oscillations.

24. Digital Signal Processing System

Answer: A typical digital signal processing system has these blocks in a chain:

1. An analog anti-aliasing low-pass filter, which removes frequencies above fs/2 so that aliasing does not occur.
2. An analog-to-digital converter (ADC): sample-and-hold, quantizer and coder.
3. A digital signal processor (a DSP chip, microprocessor or computer), which runs the algorithm (filtering, transforms, and so on).
4. A digital-to-analog converter (DAC) with a zero-order hold.
5. An analog reconstruction (anti-imaging) low-pass filter, which smooths the staircase signal and removes the spectral images.

The input x(t) is analog and the output y(t) is analog, but the processing in the middle is digital.

25. Quadrature modulation. Deviation

Answer: In quadrature modulation (QAM) two independent messages I(t) and Q(t) are sent at the same carrier frequency using two carriers with a 90° phase difference, cos and sin:

$s(t) = I(t)\cdot\cos(2\pi\cdot fc\cdot t) - Q(t)\cdot\sin(2\pi\cdot fc\cdot t)$

Because the two carriers are orthogonal, the receiver can separate them. It multiplies the received signal by $\cos(2\pi\cdot fc\cdot t)$ and by $-\sin(2\pi\cdot fc\cdot t)$ and uses low-pass filters to get I(t) and Q(t). In this way twice as much information is sent in the same bandwidth. In digital communication the pair (I, Q) is a point of the constellation, for example in 16-QAM.

Deviation. In FM and PM the frequency deviation Δf is the maximum change of the instantaneous frequency from the carrier frequency, $\Delta f = kf\cdot|x(t)|max$. The modulation index is $\beta = \Delta f/fm$, where fm is the message frequency, and the bandwidth is estimated by Carson's rule, $B \approx 2(\Delta f + fm)$.

26. Spectral characteristics

Answer: The spectrum of a signal shows how its amplitude and phase are distributed over frequency. A sinusoid has a line spectrum at ±f, and a non-periodic signal has a continuous spectrum. We get it with the Fourier transform.

For discrete-time signals the DTFT is periodic with period 2π. When a signal is sampled, its spectrum is repeated around every multiple of fs, so the sampled signal has components at $k\cdot fs \pm f$. If the spectrum is band-limited to fmax and $fs \geqslant 2\cdot fmax$, the copies do not overlap.

For a filter, the spectral characteristics are the magnitude response |H(Ω)| (the shape of the filter and its gain in dB, 20·log|H|) and the phase response θ(Ω). The magnitude response is an even function and the phase response is an odd function, and both repeat every 2π. From them we read the passband, the stopband and the cutoff frequency.

27. Modulation and demodulation

Answer: We need modulation because a baseband message (for example voice) has a low frequency and cannot be sent far by itself. By modulation we move it to a high carrier frequency, so that the antenna can be small, many channels can share the same medium (frequency multiplexing), and the noise influence is reduced.

The general modulated signal is $s(t) = A(t)\cdot\cos(2\pi\cdot fc\cdot t + \varphi(t))$. In AM the message changes $A(t)$, in PM it changes $\varphi(t)$, and in FM it changes the derivative of $\varphi(t)$. The receiver does demodulation: it selects the channel with a band-pass filter, then recovers the message, for example with an envelope detector (AM) or with a frequency discriminator (FM). In digital communication the bits are sent with ASK, FSK or PSK, and the receiver decides the bits from the received signal samples.

28. Discrete Time Systems

Answer: A discrete-time system is a system that takes an input sequence $x[n]$ and gives an output sequence $y[n] = T\{x[n]\}$. It is described by a difference equation, for example $y[n] = a\cdot y[n-1] + b\cdot x[n]$. Its basic building blocks are the adder, the multiplier and the unit delay element.

Discrete-time systems are classified as linear or non-linear, time-invariant or time-varying, causal or non-causal, stable or unstable, and static (memoryless) or dynamic. Linear time-invariant (LTI) systems are described completely by the impulse response $h[n]$, with $y[n] = x[n] * h[n]$, and by the transfer function $H(z)$. According to the impulse response, LTI systems are FIR (finite) or IIR (infinite).

29. Design of IIR filter using bilinear z-transform

Answer: The method is to design the filter in the analog domain first, and then transform it to the digital domain. The bilinear transform replaces s in the analog transfer function H(s) by

$$s = (2/T)\cdot(1 - z^{-1}) / (1 + z^{-1})$$

It maps the left half of the s-plane to the inside of the unit circle, so a stable analog filter always gives a stable digital filter. The disadvantage is that it warps the frequency axis: $\Omega a = (2/T)\cdot\tan(\omega/2)$. So the cutoff must be prewarped first.

Steps: (1) set the digital cutoff frequency; (2) prewarp it to get the analog cutoff; (3) design the analog filter (for example Butterworth) with that cutoff; (4) apply the bilinear transform to get H(z).

Example from the lecture: fs = 5 Hz (T = 0.2 s) and the desired cutoff is 2 rad/s, so the digital cutoff is 2 × 0.2 = 0.4 rad. The prewarped analog cutoff is (2/0.2)·tan(0.2) ≈ 2.03 rad/s.

30. FIR and IIR filters

Answer: An **FIR** (finite impulse response) filter uses only the present and past inputs: $y[n] = \Sigma\, b_k \cdot x[n-k]$. Its impulse response has a finite number of non-zero terms, and its transfer function has only zeros (the poles are at the origin). It is always stable, it can have exactly linear phase if the coefficients are symmetric, and it is free of limit cycles. The disadvantage is that it needs a high order, so more computation and memory.

An **IIR** (infinite impulse response) filter also uses past outputs (feedback): $y[n] = -\Sigma\, a_k \cdot y[n-k] + \Sigma\, b_k \cdot x[n-k]$. Its impulse response has an infinite length, and its transfer function has poles and zeros. It needs a much lower order for the same magnitude response, but it can be unstable, it has nonlinear phase, and it is more sensitive to coefficient quantization. A filter with only feedback is an all-pole filter.

31. Recursive smoother

Answer: A recursive smoother is a simple first-order IIR low-pass filter used to smooth a noisy signal. The new output is a weighted mix of the previous output and the new input:

$y[n] = \alpha \cdot y[n-1] + (1 - \alpha) \cdot x[n]$, with $0 < \alpha < 1$

Its transfer function is $H(z) = (1 - \alpha) / (1 - \alpha \cdot z^{-1})$, with one pole at $z = \alpha$, so it is stable. The gain at zero frequency is 1, and high frequencies are reduced, so noise is removed. If α is close to 1, the smoothing is strong but the response is slow. If α is small, the response is fast but the smoothing is weak. It needs only one multiplication memory, unlike a moving average filter (which is FIR and needs to store many past samples).

32. Group delay, linear phase, all-pass, minimum phase

Answer: Group delay is the delay of the signal envelope in the filter: $\tau g(\Omega) = -d\theta(\Omega)/d\Omega$, where θ is the phase response. The phase delay is $\tau p = -\theta(\Omega)/\Omega$.

Linear phase means $\theta(\Omega) = -\alpha \cdot \Omega$. Then the group delay is constant (equal to α samples) and all frequencies are delayed by the same time, so the signal shape is not distorted. A FIR filter has exactly linear phase when its impulse response is symmetric, $h[n] = h[N - 1 - n]$, and then $\alpha = (N - 1)/2$.

All-pass system has $|H(\Omega)| = 1$ for all frequencies, so it changes only the phase. **Minimum phase** system has all its zeros (and poles) inside the unit circle. Among all systems with the same magnitude response, it has the smallest phase lag and group delay, and its inverse is also stable.

33. Analog to Digital Conversion

Answer: An A/D converter has these main blocks: the anti-aliasing filter, the sample-and-hold circuit, the quantizer and the encoder. Two parameters decide the quality: the sampling frequency fs (must be at least 2·fmax) and the number of bits N (the resolution).

The quantizer has $L = 2^N$ levels and the step is $\Delta = R/2^N$. For example, an 8-bit converter with a 0–5 V range has 256 levels and $\Delta = 5/256 \approx 19.5$ mV. A 3-bit converter has 8 levels, and a 4-bit converter has 16 levels.

The quantization error is the difference between the true sample and the quantized level, and it is at most $\Delta/2$. The quality is measured by the dynamic range and the signal-to-quantization-noise ratio (SQNR), which is the ratio of signal power to quantization noise power and increases by about 6 dB for each added bit.

34. A fundamental definition of system stability

Answer: A system is stable if every bounded input produces a bounded output. This is called BIBO stability: if $|x[n]| \leqslant Bx < \infty$ for all n, then $|y[n]| \leqslant By < \infty$ for all n.

For an LTI discrete-time system, $y[n] = \Sigma\, h[k]\cdot x[n-k]$, so $|y[n]| \leqslant Bx\cdot\Sigma\,|h[k]|$. The output is bounded when $\Sigma\,|h[k]| < \infty$, which means the impulse response must be absolutely summable. This is the necessary and sufficient condition.

The same condition in terms of poles: a causal system is stable if all poles of H(z) are inside the unit circle. For example, $h[n] = a^n\cdot u[n]$ gives $\Sigma\,|a|^n = 1/(1-|a|)$, which is finite only for $|a| < 1$. A system with a pole on the unit circle is only marginally stable, and a system with a pole outside is unstable.

35. Sampling and Quantization

Answer: An analog signal is continuous in time and in amplitude, so it has infinite information and cannot be stored in a computer. **Sampling** reduces the time information: we keep only the values at $t = nT$, and the sampling theorem says that no information is lost if $fs \geqslant 2\cdot fmax$. The result is a discrete-time continuous-amplitude signal.

Quantization reduces the amplitude information: each sample is replaced by the nearest of $L = 2^N$ levels, with the step $\Delta = R/2^N$. The result is a discrete-time discrete-amplitude (digital) signal. Sampling is a reversible operation when the theorem is satisfied, but

quantization always adds an error (at most Δ/2) that cannot be removed, and the error is smaller when we use more bits.

36. Aliasing. Anti-aliasing in digital systems

Answer: Aliasing is a distortion that happens when the sampling rate is lower than the minimum rate, fs < 2·fmax. In the spectrum of the sampled signal the copies overlap, and high-frequency components appear as lower frequencies. A frequency f above fs/2 appears at the alias frequency |f − k·fs|, where k is the nearest integer.

Example: a 90 Hz sinusoid sampled at 100 Hz looks like a 10 Hz signal. A 20 kHz signal under-sampled at 30 kHz appears at 30 − 20 = 10 kHz. After sampling we cannot tell the real low-frequency signal from the alias.

Anti-aliasing. To be sure that aliasing does not happen, the signal is passed through an analog low-pass filter before the sampler. This anti-aliasing filter removes all frequencies above fs/2, so the signal becomes band-limited. Real filters are not ideal and have a transition band, so we usually sample a bit higher than the Nyquist rate (oversampling), which also allows a simpler, low-order analog filter.

37. Nyquist-Shannon sampling theorem

Answer: The theorem says that a band-limited signal with no frequency components above fmax can be perfectly recovered from its samples if the sampling rate is at least twice the highest frequency:

fs ⩾ 2·fmax

The minimum rate 2·fmax is called the Nyquist rate, and fs/2 is the Nyquist frequency. The reason is that sampling makes copies of the spectrum around multiples of fs, and the copies do not overlap only if fs − fmax ⩾ fmax. Then an ideal low-pass filter takes the original back. If the condition is not satisfied, aliasing occurs.

Example: x(t) = 3·sin(5000π·t + 17°) has ω = 5000π rad/s, so f = 2500 Hz and the Nyquist rate is 2 × 2500 = 5000 samples per second. Another example: a signal with components up to 3000 Hz needs at least 6000 samples per second.

38. Correlation Analysis

Answer: Correlation measures how similar two signals are when one is shifted in time. The cross-correlation of x and y is rxy[l] = Σ x[n]·y[n − l], and when y = x we get the autocorrelation rxx[l].

Properties of the autocorrelation: it is maximum at lag zero, where rxx[0] equals the energy of the signal; it is symmetric, rxx[l] = rxx[−l]; and for a periodic signal it is also periodic. Correlation is similar to convolution, but without flipping one of the signals (it is

the convolution of x with the time-reversed y).

Example: for x = {1, 2, 3}, rxx = {3, 8, 14, 8, 3}, with the maximum 14 at lag 0.

Applications: detecting a hidden periodic signal in noise, measuring the delay between two signals (radar, sonar), and finding the similarity of a signal with a known pattern.

39. Equalization filters

Answer: An equalizer is a filter that corrects the distortion made by a system or a channel, so that the total response becomes flat in magnitude and with linear phase. If the channel has the transfer function Hc(z), the ideal equalizer is the inverse, He(z) = 1/Hc(z), so that Hc(z)·He(z) ≈ 1 (or a pure delay).

In practice there are some limits. The inverse of a channel with zeros outside the unit circle (non-minimum phase) would be unstable. Also, if the channel attenuates some frequencies strongly, the equalizer amplifies the noise at these frequencies. Phase equalizers use all-pass filters to correct the group delay without changing the magnitude. Adaptive equalizers (for example the LMS algorithm) change their coefficients automatically. They are used in communication channels, audio systems and data storage.

40. Frequency Response & Transfer Function

Answer: The transfer function H(z) = Y(z)/X(z) describes the system in the z-domain. The frequency response H(Ω) is the same function evaluated on the unit circle, z = exp(jΩ), so it describes the system for sinusoidal inputs. It is also the DTFT of the impulse response h[n].

We get H(Ω) from a difference equation by taking the DTFT of each term, using the delay property x[n − M] ↔ exp(−jΩM)·X(Ω). Then H(Ω) = |H(Ω)|·exp(jθ(Ω)), where |H| is the magnitude response (gain) and θ is the phase response.

If the input is A·cos(Ω_0·n + θx), the output is |H(Ω_0)|·A·cos(Ω_0·n + θx + θ(Ω_0)), a sinusoid with the same frequency but a different amplitude and phase. The magnitude response is even, the phase response is odd, and both are periodic with period 2π.

Example: for y[n] = x[n] + x[n−1], H(Ω) = 1 + exp(−jΩ) = 2·cos(Ω/2)·exp(−jΩ/2), so |H| = 2·|cos(Ω/2)|, which is a low-pass filter.

41. Quantization Error

Answer: The quantization error is the error caused by representing a continuous-valued signal by a finite set of discrete levels: e[n] = Q(x[n]) − x[n], the difference between the quantized level and the true sample. With rounding, the error is limited to half a step, |e| ⩽ Δ/2, where Δ = R/2^N. If the signal goes outside the range, the converter clips it and a larger overload error appears.

The error is usually modeled as white noise with uniform distribution, with power $\Delta^2/12$. The signal-to-quantization-noise ratio for a full-scale sinusoid is approximately SQNR ≈ 6.02·N + 1.76 dB, so each extra bit gives about 6 dB. The more quantization levels we use, the smaller the error.

Example: with 3 bits and a 0–3 V range, $\Delta = 0.375$ V and the maximum error is 0.1875 V.

42. All-pass filters

Answer: An all-pass filter passes all frequencies with the same gain, $|H(\Omega)| = 1$, and changes only the phase. A first-order all-pass filter is

$H(z) = (z^{-1} - a) / (1 - a \cdot z^{-1})$, with $|a| < 1$

It has a pole at $z = a$ and a zero at $z = 1/a$, so the zeros are the reciprocals of the poles (mirror images about the unit circle). It is stable when the pole is inside the unit circle.

Since the magnitude is not changed, an all-pass filter does not filter anything by itself. It is used for phase equalization (to correct the group delay of another filter), for making a fractional delay, and as a building block for other filters, for example notch filters.

43. The Fourier Transform. Digital Fourier transform

Answer: The Fourier transform of a continuous signal is $X(f) = \int x(t) \cdot \exp(-j2\pi ft)\, dt$. For discrete-time signals, the DTFT is $X(\Omega) = \Sigma\, x[n] \cdot \exp(-j\Omega n)$, which is continuous and periodic in Ω. A computer cannot calculate a continuous function, so for a finite signal of N samples we use the **DFT**:

$X[k] = \Sigma$ (n = 0 to N−1) $x[n] \cdot \exp(-j2\pi kn/N)$, k = 0, 1, ..., N−1

The inverse is $x[n] = (1/N) \cdot \Sigma\, X[k] \cdot \exp(j2\pi kn/N)$. The DFT gives N equally spaced samples of the DTFT at $\Omega_k = 2\pi k/N$, which means the frequency resolution is fs/N. It is calculated quickly by the FFT algorithm, which needs about $N \cdot \log_2(N)$ operations instead of N^2. Important properties: linearity, circular shift, Parseval's theorem (the energy is the same in time and frequency), and the fact that multiplication of DFTs is circular convolution in time.

44. First-order notch filter.

Answer: A notch filter removes one single frequency (for example 50 Hz mains hum) and passes all the others almost unchanged. It is made by placing a zero on the unit circle at the unwanted frequency, which makes the gain exactly zero there, and a pole close to the zero, just inside the unit circle at the radius r (for example r = 0.95), which makes the notch narrow and keeps the gain near 1 for other frequencies.

The simplest first-order case is a notch at zero frequency (a DC blocker), with a zero at z = 1 and a pole at z = r:

$H(z) = (1 - z^{-1}) / (1 - r \cdot z^{-1})$

For a notch at a frequency Ω_0, a conjugate pair of zeros $\exp(\pm j\Omega_0)$ is needed, so the filter is second order:

$H(z) = (1 - 2\cdot\cos(\Omega_0)\cdot z^{-1} + z^{-2}) / (1 - 2r\cdot\cos(\Omega_0)\cdot z^{-1} + r^2\cdot z^{-2})$

The closer r is to 1, the narrower the notch is.

45. Second-order unity gain resonator

Answer: A resonator is a band-pass filter with a very narrow peak at one frequency Ω_0. It has a pair of complex conjugate poles near the unit circle at $r\cdot\exp(\pm j\Omega_0)$:

$H(z) = G / (1 - 2r\cdot\cos(\Omega_0)\cdot z^{-1} + r^2\cdot z^{-2})$

The gain at the peak depends on r, so G is chosen to make it exactly one (unity gain) at Ω_0: $G = (1 - r)\cdot\sqrt{(1 - 2r\cdot\cos(2\Omega_0) + r^2)}$. The 3 dB bandwidth is about $2(1 - r)$ radians, so the closer r is to 1, the narrower and sharper the peak is, but also the longer the impulse response rings. It must have $r < 1$ to be stable. It is used to select or generate a single frequency, for example a tone detector.

46. State Space.

Answer: The state-space model describes a system by first-order equations in the state vector x[n], which holds the memory (the delay elements) of the system:

$x[n+1] = A\cdot x[n] + B\cdot u[n]$

$y[n] = C\cdot x[n] + D\cdot u[n]$

Here u is the input, y is the output, and A, B, C, D are matrices. The transfer function is $H(z) = C\cdot(zI - A)^{-1}\cdot B + D$, and the poles are the eigenvalues of A, so the system is stable if all eigenvalues are inside the unit circle. The advantage of this model is that it describes the internal behavior, and it works easily for multiple-input multiple-output (MIMO) systems and for computer simulation.

Example: $y[n] = 0.5\cdot y[n-1] + u[n]$. With the state $x[n] = y[n-1]$ we get $x[n+1] = 0.5\cdot x[n] + u[n]$ and $y[n] = 0.5\cdot x[n] + u[n]$, so A = 0.5, B = 1, C = 0.5, D = 1, and $H(z) = 0.5/(z - 0.5) + 1 = z/(z - 0.5)$.

47. Transformation of Random Signals in Discrete-Time Systems.

Answer: A random signal cannot be written by an equation, so we describe it by statistics: the mean mx, the autocorrelation rxx[l] and the power spectral density Sxx(Ω), which is the DTFT of the autocorrelation. When a wide-sense stationary random signal x[n] goes through an LTI system with the impulse response h[n], the output is also random, and:

- The mean is my = mx·H(0), where H(0) is the gain at zero frequency.
- The power spectral density is Syy(Ω) = $|H(\Omega)|^2 \cdot Sxx(\Omega)$.
- The autocorrelation is ryy[l] = rxx[l] * h[l] * h[−l].
- The cross-correlation between input and output is ryx[l] = h[l] * rxx[l].

For white noise with variance σ^2, Sxx = σ^2 is flat, so the output power is $\sigma^2 \cdot \Sigma\, h^2[n]$, and the shape of the output spectrum is the shape of $|H(\Omega)|^2$. This is why a filter can reduce noise: a low-pass filter keeps only the noise in the passband.

48. High pass filters

Answer: A high-pass filter passes the high frequencies and attenuates the low frequencies. Its magnitude response is near zero at Ω = 0 and near one at Ω = π, with a cutoff frequency Ωc between the stopband and the passband. In the z-plane the zeros are placed near z = 1 (low frequencies), and the poles are placed near the high frequencies.

Example: the simple FIR filter $H(z) = (1 - z^{-1})/2$, which means y[n] = (x[n] − x[n−1])/2. Its frequency response is $H(\Omega) = (1 - \exp(-j\Omega))/2$, with $|H(\Omega)| = |\sin(\Omega/2)|$. The gain is 0 at Ω = 0 and 1 at Ω = π. In the lecture, a second-order Chebyshev type 1 high-pass filter was designed with the command cheby1(N, Rp, Wc, 'high'). High-pass filters are used to remove the DC component and slow drift from a signal, and to detect edges.

49. Low pass filters

Answer: A low-pass filter passes the low frequencies (from 0 to the cutoff frequency Ωc, the passband) and attenuates the frequencies above it (the stopband). Between them is the transition band. An ideal low-pass filter has a flat passband and a very sharp cutoff, but a real one has some ripple and a transition band. The zeros are placed near z = −1 (high frequencies) and the poles are placed near z = 1.

Example: the moving average filter $H(z) = (1 + z^{-1})/2$, which means y[n] = (x[n] + x[n−1])/2. Its response is $|H(\Omega)| = |\cos(\Omega/2)|$, which is 1 at Ω = 0 and 0 at Ω = π. Another example is the second-order Butterworth low-pass filter from the lecture, designed with the bilinear transform. Low-pass filters are used to smooth the signal and remove high-frequency noise, and the anti-aliasing and anti-imaging filters are also low-pass filters.

50. Band pass filters

Answer: A band-pass filter passes a band of frequencies between a lower cutoff Ω_1 and an upper cutoff Ω_2, and attenuates everything below Ω_1 and above Ω_2. Its important parameters are the center frequency Ω_0 and the bandwidth $BW = \Omega_2 - \Omega_1$. It can be seen as a combination of a high-pass filter and a low-pass filter, and the resonator is a special narrow case.

In the z-plane the poles are placed near the unit circle close to the center frequency Ω_0, which makes the peak, and the zeros are placed at $z = 1$ and $z = -1$, which block the DC and the highest frequency. For example, $H(z) = G\cdot(1 - z^{-2}) / (1 - 2r\cdot\cos(\Omega_0)\cdot z^{-1} + r^2\cdot z^{-2})$. Band-pass filters are used in radio receivers to select one channel, and in speech and tone detection.

51. Derive the relationship between the sampling frequency and the maximum frequency of a continuous-time signal.

Answer: Let $x(t)$ be a band-limited signal, so its spectrum $X(f) = 0$ for $|f| > f_{max}$. Sampling means we take the values at $t = nT$, where T is the sampling period and $f_s = 1/T$. We can write the sampled signal as $x(t)$ multiplied by an impulse train:

$$x_s(t) = x(t) \cdot \Sigma\, \delta(t - nT)$$

Multiplication in the time domain becomes convolution in the frequency domain, and the spectrum of an impulse train is also an impulse train at multiples of f_s. So we get

$$X_s(f) = f_s \cdot \Sigma\, X(f - k\cdot f_s)$$

This means that sampling creates copies (images) of the original spectrum around every multiple of the sampling frequency. Each copy has a width of $2\cdot f_{max}$, from $-f_{max}$ to $+f_{max}$. The first copy ($k = 1$) starts at $f_s - f_{max}$, and the baseband copy ends at f_{max}. To stop them from overlapping we need

$f_s - f_{max} \geqslant f_{max}$, so **$f_s \geqslant 2\cdot f_{max}$**

If this is true, the copies do not overlap, and a low-pass filter can take out the baseband copy and give us $x(t)$ back. If $f_s < 2\cdot f_{max}$ the copies overlap and aliasing happens. The value $2\cdot f_{max}$ is called the Nyquist rate.

52. Explain the concepts of Nyquist frequency, aliasing, and signal reconstruction

Answer: Nyquist frequency. It is half of the sampling frequency, $f_s/2$. It is the highest frequency that can be represented correctly after sampling. For digital signals the usable frequency range is from 0 to $f_s/2$. Do not mix it up with the Nyquist rate, which is $2\cdot f_{max}$ and is the minimum sampling rate we need.

Aliasing. When the sampling rate is too low (fs < 2·fmax), the spectrum copies overlap and high frequencies appear as lower frequencies. For example, a 40 Hz sinusoid sampled at 100 Hz is fine, but a 90 Hz sinusoid sampled at 100 Hz looks like a 10 Hz signal (100 − 90 = 10 Hz). After aliasing happens we cannot separate the true signal from the false one. To prevent it, an analog low-pass filter called the anti-aliasing filter is placed before the sampler, and it removes everything above fs/2.

Signal reconstruction. If the sampling theorem is satisfied, the original signal can be recovered from the samples. In theory this is done with an ideal low-pass filter with cutoff fs/2 (sinc interpolation). In practice the D/A converter converts each digital code into a voltage, a zero-order hold keeps it constant for one sampling interval (this gives a staircase signal), and then a smoothing analog low-pass filter (the anti-imaging filter) removes the high-frequency images.

53. A continuous-time signal contains frequency components up to 8 kHz. Determine the minimum sampling frequency required for accurate signal reconstruction. What happens if the signal is sampled at 12 kHz?

Answer: From the sampling theorem, fs ⩾ 2·fmax, so

fs(min) = 2 × 8 kHz = **16 kHz**.

If we sample at 12 kHz, then fs < 2·fmax, so this is under-sampling. The Nyquist frequency is now 12/2 = 6 kHz, which means only frequencies up to 6 kHz can be represented correctly. The components between 6 kHz and 8 kHz are aliased and fold back into the baseband. The aliased frequency is fa = fs − f, so an 8 kHz component appears at 12 − 8 = 4 kHz, and a 7 kHz component appears at 5 kHz. These false components mix with the real 4–5 kHz content of the signal, and we cannot remove them later. So the signal cannot be reconstructed correctly and the output is distorted.

54. Explain the fundamental principles of digital system analysis.

Answer: A digital system takes a discrete-time input sequence x[n] and gives an output sequence y[n]. In a typical digital signal processing system the analog signal first goes through an anti-aliasing filter, then the A/D converter (sampling, quantization, coding), then the digital processor, then the D/A converter, and last the reconstruction filter.

The main principles of the analysis are these:

- The system is usually described by a linear constant-coefficient difference equation, for example $y[n] = -a_1 \cdot y[n-1] + b_0 \cdot x[n] + b_1 \cdot x[n-1]$.
- We mostly work with linear time-invariant (LTI) systems, because then the system is fully described by its impulse response h[n], and the output is the convolution y[n] = x[n] * h[n].

- We analyze the system in three domains: the time domain (difference equation, impulse response), the z-domain (transfer function H(z)) and the frequency domain (frequency response H(Ω)).
- The key properties we check are linearity, time invariance, causality and stability.
- Digital systems are less sensitive to temperature and aging, and their accuracy can be chosen by the number of bits. The drawbacks are quantization error and the need for A/D and D/A converters.

55. Describe the role of the Z-transform in analyzing discrete-time systems.

Answer: The z-transform of a sequence is defined as $X(z) = \Sigma\, x[n]\cdot z^{-n}$. For discrete-time systems it plays the same role as the Laplace transform for continuous-time systems. Its main roles are:

- It turns difference equations into algebraic equations. A delay of one sample becomes multiplication by z^{-1}, so the equation is easy to solve.
- It turns convolution into multiplication: $Y(z) = H(z)\cdot X(z)$.
- It gives the transfer function $H(z) = Y(z)/X(z)$, which describes the system.
- The poles and zeros of H(z) show the stability and the behavior of the system. A causal system is stable if all poles are inside the unit circle.
- If we put $z = \exp(j\Omega)$, which is the unit circle, the z-transform becomes the frequency response of the system.
- It also helps with filter realization (direct, cascade, parallel forms) and filter design.

The region of convergence (ROC) must be given together with X(z), because it decides which sequence we mean, and it is connected to causality and stability.

56. Discuss the concepts of linearity, time invariance, causality, and stability

Answer: Linearity. A system is linear if it follows superposition: if x_1 gives y_1 and x_2 gives y_2, then $a\cdot x_1 + b\cdot x_2$ gives $a\cdot y_1 + b\cdot y_2$. For example $y[n] = 3x[n]$ is linear, but $y[n] = x[n]^2$ is not.

Time invariance. If the input is delayed by k samples, the output is delayed by the same k samples: $x[n - k]$ gives $y[n - k]$. The system does not change with time. For example $y[n] = x[n] + x[n-1]$ is time-invariant, but $y[n] = n\cdot x[n]$ is time-varying.

Causality. The output at time n depends only on the present and past inputs, not on the future ones. These are real-time systems and they can be built physically. For an LTI system this means $h[n] = 0$ for $n < 0$. For example $y[n] = x[n] + x[n-1]$ is causal, but $y[n] = x[n+1]$ is not.

Stability. A system is BIBO stable if every bounded input gives a bounded output. For an LTI system this is true when $\Sigma |h[n]| < \infty$, and for a causal system it means all the poles of H(z) are inside the unit circle.

LTI systems are the ones that are both linear and time-invariant, and they are the main class we study.

57. Explain how poles and zeros affect the behavior and frequency response of a digital system.

Answer: We can write the transfer function as $H(z) = K \cdot (z - z_1)(z - z_2)... / (z - p_1)(z - p_2)...$, where z_1, z_2 are the zeros and p_1, p_2 are the poles.

Zeros are the values of z where H(z) = 0. If a zero is on or near the unit circle at the angle Ω_0, the magnitude response has a dip or a null at the frequency Ω_0, so that frequency is attenuated. Notch filters work in this way.

Poles are the values of z where H(z) goes to infinity. A pole close to the unit circle at angle Ω_0 makes a peak (resonance) in the magnitude response near Ω_0. The closer the pole is to the unit circle, the sharper and higher the peak is, and the longer the impulse response rings.

Stability. Poles decide the stability. For a causal system all poles must be inside the unit circle. A pole on the unit circle gives a marginally stable system, and a pole outside gives an unstable one. Zeros do not affect stability.

We can also see this geometrically. At a frequency Ω the magnitude $|H(\exp(j\Omega))|$ equals the product of the distances from the point $\exp(j\Omega)$ to the zeros divided by the product of the distances to the poles. When we move along the unit circle and get close to a pole, the gain goes up, and when we get close to a zero, the gain goes down.

58. Define the transfer function of a discrete-time digital system.

Answer: The transfer function H(z) is the ratio of the z-transform of the output to the z-transform of the input, when the initial conditions are zero:

$H(z) = Y(z) / X(z)$

It is also the z-transform of the impulse response, $H(z) = \Sigma\, h[n] \cdot z^{-n}$. For a system described by a difference equation it is a rational function:

$H(z) = (b_0 + b_1 \cdot z^{-1} + ... + bM \cdot z^{-M}) / (1 + a_1 \cdot z^{-1} + ... + aN \cdot z^{-N})$

The roots of the numerator are the zeros and the roots of the denominator are the poles. The transfer function describes the system in the z-domain, and if we replace z by $\exp(j\Omega)$ we get the frequency response $H(\Omega)$.

59. Explain how the transfer function is obtained from a linear constant-coefficient difference equation using the Z-transform.

Answer: The general difference equation is

$y[n] + a_1 \cdot y[n-1] + \ldots + aN \cdot y[n-N] = b_0 \cdot x[n] + b_1 \cdot x[n-1] + \ldots + bM \cdot x[n-M]$

Step 1. Take the z-transform of both sides. Because of linearity we can do it term by term.

Step 2. Use the delay (time-shift) property: a delay by k samples becomes multiplication by z^{-k}, so $y[n-k]$ gives $z^{-k} \cdot Y(z)$, and $x[n-k]$ gives $z^{-k} \cdot X(z)$. We assume zero initial conditions.

Step 3. We get

$Y(z) \cdot (1 + a_1 \cdot z^{-1} + \ldots + aN \cdot z^{-N}) = X(z) \cdot (b_0 + b_1 \cdot z^{-1} + \ldots + bM \cdot z^{-M})$

Step 4. Divide Y(z) by X(z):

$H(z) = (b_0 + b_1 \cdot z^{-1} + \ldots + bM \cdot z^{-M}) / (1 + a_1 \cdot z^{-1} + \ldots + aN \cdot z^{-N})$

Example. For $y[n] - 0.5 \cdot y[n-1] = x[n]$, we get $Y(z)(1 - 0.5 \cdot z^{-1}) = X(z)$, so $H(z) = 1 / (1 - 0.5 \cdot z^{-1}) = z / (z - 0.5)$. There is one pole at $z = 0.5$ and one zero at $z = 0$. The pole is inside the unit circle, so the system is stable, and the impulse response is $h[n] = (0.5)^n \cdot u[n]$.

60. Discuss the relationship between the transfer function, poles, zeros, system stability, and frequency response.

Answer: All of these come from the same function H(z), so they are connected.

- **Transfer function and poles/zeros.** $H(z) = N(z)/D(z)$. The zeros are the roots of N(z) and the poles are the roots of D(z), so together with the gain K they describe H(z) completely.
- **Poles and stability.** A causal LTI system is BIBO stable if and only if all poles are inside the unit circle, $|p| < 1$. Poles on the circle give marginal stability, and poles outside give instability. FIR filters have poles only at the origin, so they are always stable.
- **Transfer function and frequency response.** The frequency response is $H(\Omega) = H(z)$ evaluated at $z = \exp(j\Omega)$, which means on the unit circle. This exists in a useful way when the unit circle is inside the ROC, which is true for stable systems. $H(\Omega)$ is also the DTFT of h[n].
- **Poles/zeros and frequency response.** Poles near the unit circle make peaks in $|H(\Omega)|$ and zeros near the unit circle make dips. The magnitude and phase responses repeat with period 2π, and for real systems the magnitude is even and the phase is odd.
- **Output for a sinusoid.** If the input is $A \cdot \cos(\Omega_0 \cdot n + \theta x)$, the output is $|H(\Omega_0)| \cdot A \cdot \cos(\Omega_0 \cdot n + \theta x + \theta)$, with the same frequency but a different amplitude and phase.

So from the pole-zero plot we can say whether the system is stable and what its filter shape will be (low-pass, high-pass, band-pass or band-stop).

61. Sampling of an Analog Signal (f = 5; fs = 20; t = 10 s)

```
t = 0 : 1/20 : 10;
x = sin(2 pi 5 * t);
plot(t, x);
```

62. Quantization of a Signal (x = sin(0.2·π·n), n = 50)

```
n = 0 : 50;
x = sin(0.2 pi n);
xq = round(x * 2) / 2;
plot(n, x, n, xq);
```

63. Write a MATLAB program to generate a random signal and perform its discretization using different sampling frequencies. Compare the sampled signals and explain the influence of sampling frequency on signal representation.

```
% 1. Random Signal
x = rand(1, 100);
% 2. Low Sampling Frequency (fs = 10)
x1 = x(1:10:end);
% 3. High Sampling Frequency (fs = 2)
x2 = x(1:2:end);
% 4. Compare plots
subplot(2,1,1);
plot(x1);
title('Low fs');
subplot(2,1,2);
plot(x2);
title('High fs');
```

64. Generate a random signal using MATLAB.

```
% random signal
x = randn(1, 100);
plot(x);
title('Random Signal');
xlabel('Sample');
ylabel('Amplitude');
```

65. Sample the signal at 20 Hz, 50 Hz, and 100 Hz.

```
% 1. Create a base signal
n = 0:200;
x = sin(0.1*n) + 0.2*cos(0.3 * n);
% 2. Sample at different rates (Basic indexing)
x20 = x(1:10:end); % Low sampling
x50 = x(1:4:end); % Medium sampling
x100 = x(1:2:end); % High sampling
% 3. Plots
subplot(3,1,1); plot(x20); title('Sampled at 20 Hz');
subplot(3,1,2); plot(x50); title('Sampled at 50 Hz');
subplot(3,1,3); plot(x100); title('Sampled at 100 Hz');
```

66. Display the original and sampled signals.

```
% 1. Original Signal
n = 0:100;
x = sin(0.2 * n);
% 2. Sampling
x20 = x(1:10:end);
x50 = x(1:4:end);
x100 = x(1:2:end);
% 3. Plots
```

subplot(4,1,1); plot(x); title('Original Signal');

subplot(4,1,2); plot(x20); title('Sampled at 20 Hz');

subplot(4,1,3); plot(x50); title('Sampled at 50 Hz');

subplot(4,1,4); plot(x100); title('Sampled at 100 Hz');

67. Use a Simulink model for generate a random signal and perform its discretization using different sampling frequencies

1. Lazım olan blokların siyahısı (Library Browser)

- **Random Number** *(Sources bölməsindən)* – Təsadüfi siqnal generasiya etmək üçün.
- **Zero-Order Hold** *(Discrete bölməsindən)* – Siqnalı diskretləşdirmək üçün (Bizə **3 ədəd** lazımdır).
- **Scope** *(Sinks bölməsindən)* – Qrafikləri görmək üçün ekran (Giriş sayını **3** edirik).

---

2. Blokların sazlanması (Üzərinə iki dəfə klikləyin)

- **Random Number:**
  - `Sample time` = `0` (Bu, bizim orijinal, kəsilməz təsadüfi siqnalımızdır).
- **1-ci Zero-Order Hold (20 Hz üçün):**
  - `Sample time` = `1/20`
- **2-ci Zero-Order Hold (50 Hz üçün):**
  - `Sample time` = `1/50`
- **3-cü Zero-Order Hold (100 Hz üçün):**
  - `Sample time` = `1/100`

---

3. Blokların birləşdirilməsi sxemi (Simulink Layout)

Blokları siçan (mouse) vasitəsilə sadəcə bu şəkildə bir-birinə bağlayın:

+---> [Zero-Order Hold (1/20)] ----> Scope Giriş 1 (20 Hz)

|

[Random Number] ------+---> [Zero-Order Hold (1/50)] ----> Scope Giriş 2 (50 Hz)

|

+---> [Zero-Order Hold (1/100)] ---> Scope Giriş 3 (100 Hz)

68. using Quantiazer create simulink model for pulse and sine wave.

1. Lazım olan bloklar (Library Browser)

- **Sine Wave** *(Sources bölməsindən)* – Sinus dalğası yaratmaq üçün.
- **Pulse Generator** *(Sources bölməsindən)* – İmpuls siqnalı yaratmaq üçün.
- **Quantizer** *(Discontinuities bölməsindən)* – Siqnalı səviyyələrə (pillələrə) bölmək üçün (**2 ədəd**).
- **Scope** *(Sinks bölməsindən)* – Nəticələri görmək üçün ekran (**2 ədəd**).

---

2. Blokların sazlanması (Üzərinə iki dəfə klikləyin)

- **Quantizer (Hər ikisi üçün):** `Quantization interval` hissəsinə **`0.5`** yazın. Bu, siqnalı hər 0.5 vahiddən bir pilləkən formasına salacaq.
- **Sine Wave & Pulse Generator:** Standart (default) tənzimləmələrdə qala bilər.

---

3. Blokların birləşdirilməsi sxemi (Simulink Layout)

Sxemi çox rahat şəkildə alt-alta iki ayrı xətt kimi birləşdirirsiniz:

Sinus dalğası üçün:

[Sine Wave] --------> [Quantizer (0.5)] --------> [Scope 1]

İmpuls siqnalı üçün:

[Pulse Generator] --> [Quantizer (0.5)] --------> [Scope 2]

69. restore signals using first order hold

1. Lazım olan bloklar (Library Browser)

- **Sine Wave** *(Sources bölməsindən)* – Əsas analoq siqnalımız.
- **Zero-Order Hold** *(Discrete bölməsindən)* – Siqnalı öncə diskretləşdirmək üçün (Məsələn, `Sample time = 0.1`).
- **First-Order Hold** *(Discrete bölməsindən)* – Siqnalı xətti şəkildə bərpa etmək üçün (Məsələn, `Sample time = 0.1`).
- **Scope** *(Sinks bölməsindən)* – Orijinal və bərpa olunmuş siqnalları müqayisə edib görmək üçün ekran (Giriş sayını **2** edin).

---

2. Blokların birləşdirilməsi sxemi (Simulink Layout)

Blokları çox sadə şəkildə belə birləşdirirsiniz:

+----------> Scope Giriş 1 (Original)

|

[Sine Wave] -+---> [Zero-Order Hold] ---> [First-Order Hold] ---> Scope Giriş 2 (Restored)

70. Write a MATLAB code for stability

% 1. Define coefficients

num = [1]; % Numerator: 1

den = [1, 3, 2]; % Denominator: s^2 + 3s + 2

% 2. Find the poles (roots of the denominator)

poles = roots(den);

% 3. Check stability for continuous system (s-domain)

% For stability, the real parts of all poles must be negative (< 0)

if all(real(poles) < 0)

disp('The system is stable.');

else

disp('The system is unstable.');

end

71.

Consider the digital system:

$$H(z) = \frac{1 + 0.5z^{-1}}{1 - 0.7z^{-1}}$$

% 1. Define coefficients

num = [1, 0.5]; % Coefficients for 1 + 0.5z^-1

den = [1, -0.7]; % Coefficients for 1 - 0.7z^-1

% 2. Find the roots of the denominator (poles)

poles = roots(den);

% 3. Stability check for digital system (z-domain)

% The magnitude of the poles must be less than 1 (|z| < 1)

if all(abs(poles) < 1)

disp('The digital system is stable.');

else

disp('The digital system is unstable.');

end

72. Analyze the stability of a digital control system using the Nyquist criterion in Matlab.

Consider the discrete-time open-loop transfer function:

$$G(z) = \frac{0.5}{z - 0.6}$$

% 1. Define numerator and denominator

num = [0.5];

den = [1, -0.6];

% 2. Create the digital open-loop transfer function

G = tf(num, den, -1);

% 3. Plot the Nyquist diagram

figure;

nyquist(G);

grid on;

% 4. Check closed-loop stability mathematically

T = feedback(G, 1);

poles = pole(T);

% If the maximum pole magnitude is less than 1, it's stable

if max(abs(poles)) < 1

disp('System is STABLE (Does not encircle -1)');

else

disp('System is UNSTABLE (Encircles -1)');

end

73.Analysis of a Digital Control System Using the Routh–Hurwitz Criterion in MATLAB

Determine the stability of a digital control system using the Routh–Hurwitz criterion.

% 1. Routh-Hurwitz requires Bilinear Transformation: z = (1+w)/(1-w)

% Example for (z - 0.5 = 0) becomes: 1.5*w + 0.5 = 0

% 2. Enter w-domain coefficients

coeff = [1.5, 0.5];

% 3. Routh-Hurwitz rule: All coefficients must be positive (>0)

if all(coeff > 0)

disp('System is STABLE');

else

disp('System is UNSTABLE');

end

74.

Given the characteristic polynomial of a system:

$$2s^5 + 4s^4 + 3s^3 + 2s^2 + s + 2 = 0.$$

Determine whether the system is stable using the Routh–Hurwitz stability criterion.

% 1. Define polynomial coefficients

coeff = [2, 4, 3, 2, 1, 2];

% 2. Find roots (poles) of the system

poles = roots(coeff);

% Display the calculated roots in the command window

disp('The roots (poles) of the system are:');

disp(poles);

% 3. Check stability: All roots must have negative real parts

if all(real(poles) < 0)

disp('System is STABLE');

else

disp('System is UNSTABLE');

end

75.Given the characteristic polynomial of a system:

s^5+14s^4+2s^3+5s^2+s+5=0.

Determine whether the system is stable using the Mikhailov stability criterion.

% 1. Define polynomial coefficients for s^5 + 14s^4 + 2s^3 + 5s^2 + s + 5 = 0

coeff = [1, 14, 2, 5, 1, 5];

% 2. Find the roots (poles) of the system

poles = roots(coeff);

% Display the calculated roots in the command window

disp('The roots (poles) of the system are:');

disp(poles);

% 3. Check stability: All roots must have negative real parts

if all(real(poles) < 0)

disp('System is STABLE');

else

disp('System is UNSTABLE');

end

76. Given the transfer function of a system:

$$W(s) = \frac{s+1}{3s^3 + 3s^2 + 3s + 1}$$

Determine whether the system is stable using the Nyquist plote .

% 1. Define numerator and denominator coefficients

num = [1, 1]; % Numerator: s + 1

den = [3, 3, 3, 1]; % Denominator: 3s^3 + 3s^2 + 3s + 1

% 2. Create the transfer function

G = tf(num, den);

% 3. Plot the Nyquist diagram

figure;

nyquist(G);

grid on;

title('Nyquist Plot');

```
% 4. Check closed-loop stability mathematically
T = feedback(G, 1);
poles = T.Denominator{1}; % Get closed-loop poles
closed_loop_poles = roots(poles);
if all(real(closed_loop_poles) < 0)
disp('System is STABLE (Does not encircle -1)');
else
disp('System is UNSTABLE (Encircles -1)');
end
```

77. Using the characteristic polynomial of a system:

$$W(s) = \frac{10s + 1}{8s^3 + 6s^2 + 3s + 1}$$

Determine whether the system is stable using the Routh–Hurwitz stability criterion.

```
% 1. Define the denominator coefficients for 8s^3 + 6s^2 + 3s + 1 = 0
coeff = [8, 6, 3, 1];
% 2. Find the roots (poles) of the system
poles = roots(coeff);
% Display the calculated roots in the command window
disp('The roots (poles) of the system are:');
disp(poles);
% 3. Check stability: All roots must have negative real parts
if all(real(poles) < 0)
disp('System is STABLE');
else
disp('System is UNSTABLE');
end
```